

Overdex Battle Architecture

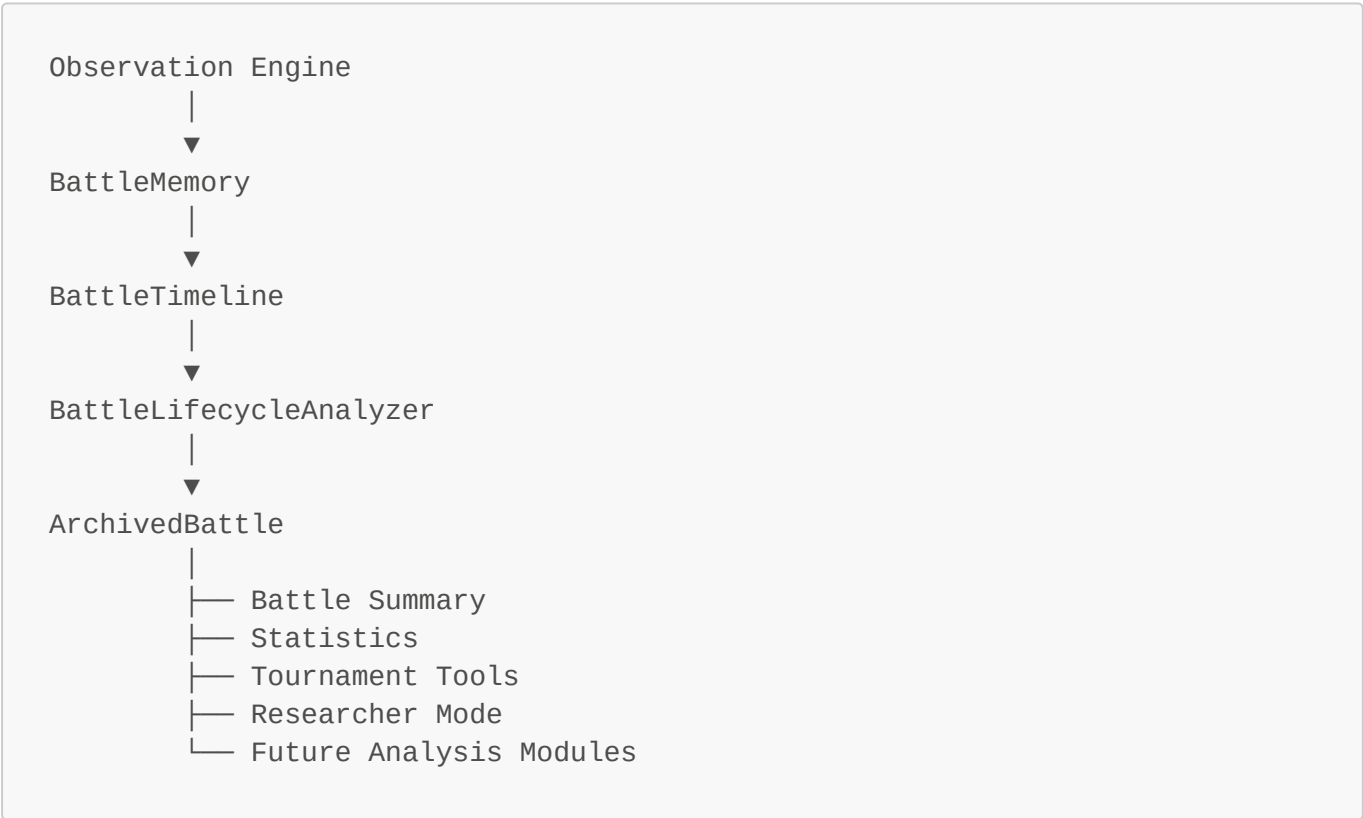
This document describes the high-level data flow and system hierarchy for the Overdex battle support system.

Core Architectural Principle

Observe first. Infer second. Present last.

- Every layer has a single responsibility.
 - Observation produces facts.
 - Inference derives conclusions.
 - Presentation communicates those conclusions to the player.
 - No layer should assume the responsibilities of another.
-

High-Level Data Flow



Architectural Components

1. Observation Engine

The entry point for every observation source.

Examples include:

- Screen Capture
- OCR
- Audio
- Manual Input
- Prototype Sensors
- Future Observation Sources

Its only responsibility is to detect facts and produce standardized observations.

It never makes gameplay decisions.

2. BattleMemory

The authoritative source of truth for the current battle.

BattleMemory represents the present.

It stores the current battle state:

- Active Pokémon
- Health
- Energy
- Shields
- Timers
- Other live observations

Whenever the observed state changes, BattleMemory records the fact.

3. BattleTimeline

A chronological journal of immutable **BattleEvent** objects.

Every observation becomes an event.

The timeline is the canonical record of everything that occurred during a battle.

Events are never rewritten after they have been recorded.

4. BattleLifecycleAnalyzer

Determines when a battle begins and ends.

This component consumes observations from BattleMemory and BattleTimeline.

It does not create observations.

It evaluates accumulated evidence to determine battle lifecycle events.

5. ArchivedBattle

A completed, immutable snapshot of a battle.

It contains:

- Complete BattleTimeline
- Timing metadata
- Battle metadata
- Lifecycle information

ArchivedBattle is the permanent historical record.

Once created, it is never modified.

6. Battle History

The management layer for archived battles.

Battle History exists to provide data to downstream analysis modules.

It does not reinterpret history.

Downstream Consumers

Every downstream system consumes the same ArchivedBattle.

Examples include:

- Battle Summary
- Statistics
- Tournament Tools
- Researcher Mode
- Future analysis modules

Interpretation belongs to these consumers, not to the observation system.

Design Philosophy

Facts before conclusions.

Overdex functions as the player's second brain.

Its primary responsibility is to capture objective facts accurately.

Intelligence should emerge from those facts rather than replace them.

Battle Lifecycle

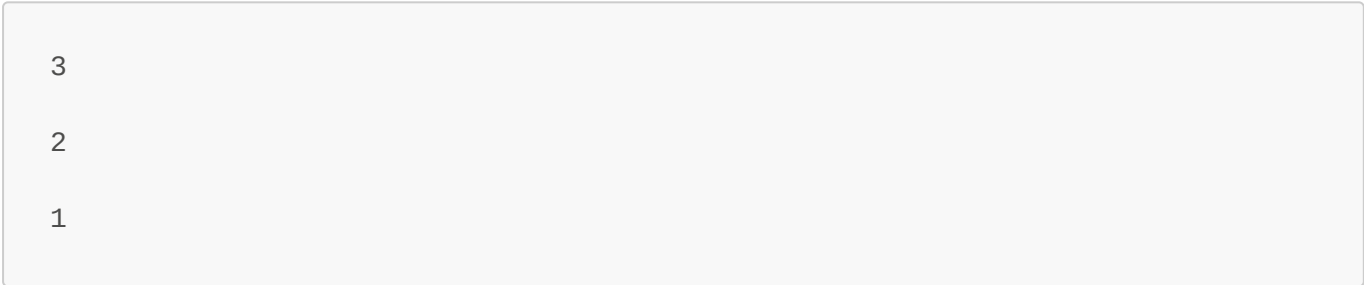
The Observation Engine records facts only.

BattleMemory and BattleTimeline never determine battle outcomes.

Their responsibility ends with recording observations.

Battle Start

A battle officially begins when Pokémon GO displays the battle countdown:



3
2
1

This is considered the highest-confidence indication that a battle has started.

Battle End

A battle may terminate in one of three states:

- Victory
- Defeat
- Abandonment

Victory and defeat should be determined using multiple observations whenever possible.

Possible observations include:

- Victory / Defeat screen
- Victory / Defeat audio
- Timeline consistency
- Remaining Pokémon
- Future observation sources

No single observation should ever be considered absolute truth.

Observation Degradation

Overdex should continue operating whenever possible.

If Screen Capture is lost, the Observation Engine should continue using any remaining observation sources.

Examples include:

- Audio
- Manual input
- Future sensors

Loss of one observation source reduces confidence but should not immediately terminate battle analysis.

Every observation source is optional.

Confidence reflects the amount and quality of available evidence.

Architectural Guarantees

- Observation produces facts.
- BattleMemory preserves the present.
- BattleTimeline preserves history.
- ArchivedBattle preserves completed battles.
- Downstream systems perform interpretation.
- The Overlay speaks last.

Every layer has one responsibility.

Every responsibility has one owner.